



Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Santa Fe

Analítica de datos y herramientas de inteligencia artificial II (Gpo 502)

Entrega de reto

Alumno:

Francisco Antonio López Ricardez - A01737275

Diciembre 05, 2025

Introducción

Este proyecto tiene como objetivo desarrollar un agente autónomo capaz de jugar Super Mario Bros mediante técnicas avanzadas de aprendizaje por refuerzo profundo. El propósito central consiste en entrenar un sistema que, a partir de la observación directa del entorno visual, aprenda a desplazarse eficientemente a través del nivel SuperMarioBros-1-1, maximice su progreso y tome decisiones óptimas en cada estado del juego.

El entorno seleccionado ofrece observaciones de alta dimensionalidad basadas en secuencias de imágenes, un espacio de acciones discreto y dinámicas altamente dependientes del tiempo. Este tipo de problema requiere combinar percepción visual, toma de decisiones secuenciales y capacidad de adaptación, por lo que resulta adecuado el uso de deep learning.

Para abordar este reto se implementó un agente basado en Deep Q-Network y más específicamente DQN, junto con una red neuronal convolucional encargada de procesar los frames del juego. Asimismo, se utilizaron técnicas estandarizadas en entornos tipo Atari, como frame stacking, reducción de dimensionalidad, normalización y sincronización periódica de una red objetivo para estabilizar el entrenamiento.

El desarrollo del sistema abarca desde la definición del problema como un Proceso de Decisión de Markov, hasta la construcción del pipeline de interacción con el entorno, almacenamiento de experiencias mediante un Replay Buffer y un ciclo de entrenamiento iterativo que permite al agente mejorar progresivamente su desempeño. Además, se estableció un esquema experimental para monitorear métricas clave como recompensa acumulada, estabilidad de la estimación del valor Q y duración de los episodios.

En conjunto, el proyecto implementa una solución completa y funcional para el control autónomo de un agente en un entorno visual complejo, demostrando la capacidad de técnicas de aprendizaje profundo para adquirir comportamientos avanzados sin necesidad de supervisión explícita.

Planteamiento

El reto abordado en este proyecto consiste en diseñar un sistema capaz de controlar autónomamente a un agente dentro del entorno SuperMarioBros-1-1-v0. El objetivo es que el agente logre avanzar de manera eficiente, evitar obstáculos, optimizar su desplazamiento y, en la medida de lo posible, completar el nivel. Para ello, debe tomar decisiones en tiempo real basadas exclusivamente en la información visual que recibe del entorno.

El comportamiento del juego puede describirse como un Proceso de Decisión de Markov, definido por los siguientes elementos:

- **Estados:** representados por una secuencia de imágenes en escala de grises normalizadas y redimensionadas, las cuales reflejan la vista actual del entorno. Se utiliza un stack de cuatro frames consecutivos para capturar el componente temporal del movimiento.
- **Acciones:** dada la naturaleza del nivel y el alcance del proyecto, el espacio de acciones se reduce a dos combinaciones discretas: avanzar a la derecha, y avanzar a la derecha con salto. Este conjunto simplificado es suficiente para permitir que el agente atraviese el nivel.
- **Recompensas:** el agente recibe recompensas densas asociadas al progreso horizontal y recompensas más significativas cuando alcanza la meta. De igual forma, se penaliza la pérdida de vida o falta de progreso durante varios pasos.

- **Transiciones:** determinadas por la dinámica del juego, donde cada acción modifica el estado y desencadena nuevas observaciones.
- **Estados terminales:** definidos cuando Mario pierde todas sus vidas, cae en un vacío o alcanza la bandera del nivel.

Este entorno presenta varias características que lo convierten en un caso representativo para aplicar aprendizaje por refuerzo profundo:

1. **Alta dimensionalidad de observaciones:** las entradas son imágenes de 84×84 píxeles procesadas en tiempo real, lo que requiere una red neuronal con capacidad de extracción de características visuales.
2. **Recompensas escasas y ruidosas:** aunque existen señales proporcionales al avance, el objetivo final solo se alcanza al completar el nivel, lo que demanda estrategias de aprendizaje que puedan lidiar con retroalimentación limitada.
3. **Dinámicas complejas y dependientes del tiempo:** enemigos, plataformas, obstáculos y huecos introducen variabilidad significativa en las transiciones.
4. **Acciones discretas y secuenciales:** el control requiere seleccionar acciones apropiadas en una secuencia prolongada, donde errores tempranos pueden arruinar el episodio completo.

Por lo anteriormente mencionado se determinó que el enfoque más adecuado es el uso de Deep Reinforcement Learning, en particular algoritmos basados en Deep Q-Networks, los cuales han demostrado resultados sobresalientes en entornos similares como Atari.

Pipeline de datos y preprocesamiento

A diferencia de los proyectos tradicionales de aprendizaje supervisado, donde los datos se encuentran disponibles desde el inicio, en este proyecto las experiencias del agente se generan dinámicamente mediante su interacción con el entorno. Por lo tanto, el diseño del pipeline de datos es un componente crítico para garantizar que el agente reciba información útil, estable y representativa para aprender una política óptima.

El proceso completo de flujo de datos puede dividirse en tres etapas principales:

- generación de experiencias
- preprocesamiento de observaciones
- almacenamiento estructurado en un Replay Buffer.

Generación de experiencias

En cada paso del entrenamiento, el agente observa el estado actual del entorno, selecciona una acción según su política balanceando exploración y explotación, y recibe como respuesta:

- el nuevo estado del entorno,
- la recompensa obtenida,
- un indicador de finalización del episodio,
- metadatos adicionales del juego.

Este ciclo permite que las transiciones se recopilen de manera continua, formando la base del conjunto de datos con el que el agente entrena.

Cada una de estas transiciones se guarda para entrenamiento futuro, manteniendo la coherencia temporal solo en el nivel del *batching*, pero rompiéndola al momento de entrenar, lo que mejora la estabilidad del algoritmo DQN.

Preprocesamiento de observaciones

Las imágenes originales del juego tienen alta resolución y color, lo que implica un costo computacional significativo y dificulta el aprendizaje de representaciones relevantes. Por ello, se aplica un conjunto de transformaciones inspiradas en los trabajos originales de DeepMind para entornos Atari:

- **Conversión a escala de grises:** Reduce la dimensionalidad y elimina información de color irrelevante para la toma de decisiones.
- **Normalización de valores de píxeles:** Se escalan los valores a un rango de 0 a 1, facilitando la estabilidad del entrenamiento de la red neuronal.
- **Redimensionamiento a 84×84 píxeles:** Mantiene suficiente detalle visual mientras reduce el costo computacional, permitiendo entrenar más rápidamente y con menos memoria.
- **Frame Stacking con acumulación de 4 frames consecutivos:** Dado que una imagen estática no contiene información sobre velocidades o tendencias de movimiento, se apilan los últimos cuatro cuadros del juego. Esto introduce un sentido de temporalidad y permite que la red convolucional identifique dinámicas como saltos, desplazamientos y aparición de enemigos.

El resultado final del preprocesamiento es un tensor de forma [4, 84, 84] que representa el estado actual del entorno de manera compacta y significativa.

Almacenamiento en Replay Buffer

Una vez generadas y preprocesadas las observaciones, cada transición se almacena en un Replay Buffer, que actúa como una memoria de capacidad fija donde se guardan las experiencias más recientes del agente.

Las ventajas de utilizar un Replay Buffer son:

- **Descorrelación de datos:** Las transiciones consecutivas en un videojuego están altamente correlacionadas. Muestrear de manera aleatoria rompe esta correlación y mejora la estabilidad del aprendizaje.
- **Reutilización de experiencias:** Cada experiencia puede contribuir a múltiples actualizaciones del modelo, incrementando la eficiencia de los datos generados.
- **Control del flujo de entrenamiento:** Se utiliza un periodo inicial de warmup antes de iniciar el entrenamiento, garantizando que el buffer contenga experiencias variadas desde el inicio.

El almacenamiento está limitado a un número fijo de elementos, lo que permite un uso eficiente de memoria. Cuando el buffer se llena, las experiencias más antiguas se descartan, manteniendo solo información reciente y relevante.

Integración del pipeline con el entrenamiento

El pipeline descrito se integra de forma directa con el agente y la red neuronal:

- El agente obtiene estados procesados desde el pipeline.
- Los guarda en el buffer conforme interactúa con el entorno.
- Durante el entrenamiento, extrae minibatches aleatorios del buffer.
- Esos minibatches alimentan la red neuronal online para actualizar los valores Q.
- Periódicamente, los parámetros se sincronizan con la red target.

Este proceso se repite durante miles de pasos, permitiendo que el sistema aprenda progresivamente una política más eficiente para completar el nivel.

Metodología

La metodología implementada sigue un ciclo iterativo en el que el agente interactúa con el entorno, recopila experiencias, actualiza su política mediante aprendizaje supervisado sobre valores Q estimados y evalúa periódicamente su desempeño.

El diseño metodológico se divide en cinco componentes principales:

- interacción con el entorno
- almacenamiento de experiencias
- aprendizaje con Double DQN
- evaluación continua
- gestión de checkpoints para reproducibilidad.

Interacción con el entorno

En cada paso del ciclo, el agente observa el estado actual del juego mediante los frames preprocesados y selecciona una acción utilizando una política ϵ -greedy, donde ϵ representa la tasa de exploración.

- **Exploración:** el agente elige acciones aleatorias con probabilidad ϵ para descubrir nuevos estados.
- **Explotación:** con probabilidad $1-\epsilon$, el agente selecciona la acción con mayor valor Q estimado por la red neuronal.

El uso de la política ϵ -greedy permite equilibrar adecuadamente la búsqueda de nuevas estrategias con la explotación del conocimiento adquirido. Tras ejecutar la acción seleccionada, el entorno proporciona una nueva observación, una recompensa y un indicador de finalización del episodio. Esta información forma parte del conjunto de datos sobre el cual el agente aprende.

Almacenamiento estructurado de experiencias

Cada transición es almacenada en un Replay Buffer de capacidad finita. Este componente cumple funciones esenciales:

- **Descorrelación de experiencias:** romper la naturaleza secuencial de las observaciones.
- **Reutilización eficiente:** las mismas experiencias pueden contribuir a múltiples actualizaciones.
- **Diversidad:** se garantiza que el agente no entrene únicamente sobre secuencias recientes o repetitivas.

Antes de iniciar el entrenamiento formal, se ejecuta un periodo de *warm up* donde el agente sólo explora, permitiendo poblar el buffer con transiciones variadas.

Aprendizaje con Double Deep Q-Network

El núcleo de la metodología de entrenamiento se basa en Double DQN, al aplicar esta versión de DQN se reduce la sobreestimación de los valores Q. Este enfoque utiliza dos redes convolucionales:

Red Online:

- Predice los valores Q actuales.
- Se actualiza en cada iteración de entrenamiento.

Red Target:

- Estabiliza el aprendizaje proporcionando valores objetivos más consistentes.
- Se sincroniza periódicamente con los parámetros de la red online.

Proceso de actualización

- Se obtiene un mini batch aleatorio del Replay Buffer.
- La red online predice el valor Q actual para cada estado del batch.
- La red online también se usa para seleccionar las mejores acciones futuras.
- La red target se utiliza para evaluar los valores Q de esas acciones, generando los valores objetivos.
- El error entre los valores predictivos y los valores objetivo se minimiza mediante Smooth L1 Loss.
- El optimizador actualiza únicamente la red online.
- Cada cierto número de pasos, la red target se sincroniza.

Este mecanismo desacopla la acción seleccionada del valor con el que se evalúa, reduciendo la inestabilidad típica del aprendizaje por refuerzo.

Evaluación y registro de métricas

Durante el entrenamiento, el sistema monitorea continuamente indicadores como:

- Recompensa total por episodio.
- Duración del episodio.
- Pérdida de entrenamiento.

- Valor Q promedio.
- Señales de éxito como por ejemplo, si se alcanzó la bandera del nivel.

Estas métricas se registran mediante un módulo dedicado a la visualización, lo cual permite detectar tendencias, inestabilidades y puntos de mejora en la política del agente. La evaluación interna se realiza sin detener el entrenamiento, aplicando promedios móviles para suavizar curvas y facilitar su interpretación.

Gestión de checkpoints y reproducibilidad

Para garantizar reproducibilidad y permitir tanto demostración como reentrenamiento incremental, el sistema genera checkpoints periódicos que incluyen:

- Los parámetros de la red online.
- La tasa actual de exploración (ϵ).
- El número de pasos globales del entrenamiento.

Estos checkpoints pueden cargarse posteriormente para:

- Continuar el entrenamiento desde el estado previo.
- Utilizar el modelo exclusivamente en modo inferencia, reproducido desde `replay.py`
- Analizar versiones anteriores del agente y compararlas con iteraciones posteriores.

Integración metodológica

El flujo metodológico completo puede representarse de la siguiente manera:

1. Inicializar el entorno y el agente.
2. Ejecutar un episodio:
 - a. Obtener el estado actual.
 - b. Seleccionar acción vía política ϵ -greedy.
 - c. Ejecutar acción en el entorno.
 - d. Registrar transición en el Replay Buffer.
 - e. Entrenar si el sistema ha superado el periodo de warmup.
3. Al finalizar un episodio, registrar métricas clave.
4. Periódicamente sincronizar la red target y guardar checkpoints.
5. Repetir durante miles de episodios hasta observar convergencia o comportamiento satisfactorio.

Arquitectura del modelo

La arquitectura central del agente se basa en una Red Neuronal Convolutiva diseñada para procesar secuencias de imágenes del entorno de Super Mario Bros. Esta red forma parte de la implementación de Double Deep Q-Network y es responsable de estimar el valor Q para cada acción disponible, permitiendo que el agente seleccione la acción más adecuada en cada estado del juego.

La arquitectura implementada sigue un diseño similar al utilizado en los trabajos de DeepMind para Atari, optimizado para funcionar sobre observaciones visuales reducidas y normalizadas.

Entrada del modelo

La entrada consiste en un tensor de dimensiones [4,84,84] que corresponde a:

- 4 frames consecutivos para el frame stacking.
- convertidos a escala de grises.
- normalizados a valores entre 0 y 1.
- redimensionados a 84×84 píxeles.

Este formato permite capturar información temporal suficiente para interpretar el movimiento de Mario, la velocidad relativa de enemigos y la inercia del jugador.

Arquitectura convolucional

La red está compuesta por tres capas convolucionales principales:

- **Conv1**
 - 32 filtros
 - Tamaño de kernel: 8×8
 - Stride: 4
 - Función de activación: ReLU
 - El propósito de esta primera capa convolucional es la extracción inicial de características espaciales amplias.
- **Conv2**
 - 64 filtros
 - Kernel: 4×4
 - Stride: 2
 - Activación: ReLU
 - El propósito de esta segunda capa es la detección de patrones más complejos (bordes, movimiento, obstáculos).
- **Conv3**
 - 64 filtros
 - Kernel: 3×3
 - Stride: 1
 - Activación: ReLU

- El propósito de esta tercera capa es el refinamiento de características de alto nivel necesarias para la toma de decisiones.

Estas capas reducen progresivamente la dimensionalidad espacial, incrementando la profundidad y abstracción de la representación interna.

Capas densas y estimación del valor Q

Tras la etapa convolucional, la salida se aplanan en un vector que alimenta dos capas densas:

- **Capa fully-connected intermedia**
 - 512 unidades
 - Activación: ReLU
 - Propósito: generar una representación compacta del estado, integrando la información espacial y temporal.
- **Capa fully-connected de salida**
 - Número de neuronas igual al número de acciones disponibles.
 - salida lineal.
 - Cada neurona representa el valor Q estimado para una acción específica.

Esta última capa es la que se utiliza para seleccionar acciones mediante políticas ϵ -greedy o greedy en modo inferencia.

Arquitectura Double DQN: Red Online y Red Target

Para mitigar problemas de sobreestimación y mejorar la estabilidad del aprendizaje, se implementa la arquitectura Double DQN, que introduce dos redes simétricas:

Red Online

- Realiza la predicción de los valores Q actuales.
- Se actualiza en cada paso de entrenamiento.
- Es la red utilizada para seleccionar la acción con mejor valor Q en el estado siguiente.

Red Target

- Mantiene parámetros congelados durante intervalos específicos.
- Evalúa el valor Q de la acción seleccionada por la red online.
- Se sincroniza periódicamente copiando los pesos de la red online.

Este desacoplamiento entre selección y evaluación de acciones corrige la tendencia a sobreestimar valores Q y reduce oscilaciones durante el entrenamiento.

Función de pérdida y optimización

El entrenamiento se realiza minimizando la Smooth L1 Loss entre los valores Q predichos y los valores objetivo generados por la red target. Esta función es adecuada para entornos de RL debido a su robustez ante valores atípicos y su estabilidad en presencia de recompensas ruidosas.

El optimizador utilizado es Adam, seleccionado por su capacidad de adaptarse dinámicamente a los gradientes y su buen desempeño en modelos convolucionales con datos no estacionarios.

Justificación de la arquitectura

La combinación de CNNs profundas y Double DQN es ampliamente utilizada en tareas de control basadas en visión por las siguientes razones:

- **Procesamiento visual eficiente:** las convoluciones permiten extraer representaciones buenas sin necesidad de ingeniería de características manual.
- **Capacidad de manejar entornos complejos:** los videojuegos como Mario presentan obstáculos, movimientos rápidos y variabilidad en los escenarios.
- **Estabilidad y robustez:** Double DQN reduce inestabilidades típicas de DQN puro.

En conjunto, el modelo implementado constituye un sistema altamente adecuado para el control autónomo en entornos visuales dinámicos.

Diseño experimental

El agente fue entrenado en el entorno SuperMarioBros-1-1-v0, un escenario ideal para evaluar comportamientos secuenciales en un ambiente visual complejo. Se utilizaron solo dos acciones: avanzar y avanzar con salto, con el fin de mantener el problema dentro de un espacio de decisiones manejable y enfocado en los comportamientos esenciales necesarios para superar el nivel.

El entorno fue transformado mediante una serie de wrappers estandarizados, que permiten reducir complejidad visual, mejorar la estabilidad del entrenamiento y acelerar el procesamiento. Entre ellos se incluyen:

- Conversión a escala de grises.
- reducción de la resolución a 84×84.
- salto de frames para suavizar la dinámica del entorno.
- normalización de valores.
- apilamiento de cuatro frames para incorporar información temporal.

Este conjunto de transformaciones aproxima el preprocesamiento utilizado en los trabajos originales de DQN con Atari.

Configuración del agente

El agente emplea una política ϵ -greedy con exploración inicial completa que se reduce gradualmente conforme avanza el entrenamiento. Esta política permite que el agente explore ampliamente el entorno en sus primeras etapas y posteriormente concentre su comportamiento en las acciones que le han dado mejores resultados.

Sus parámetros principales son:

- **ϵ inicial:** 1.0
- **ϵ mínimo:** 0.1
- **Tasa de decaimiento:** 0.99999975
- **Replay Buffer:** 100,000 transiciones
- **Batch size:** 32
- **Warmup:** 10,000 pasos antes de iniciar entrenamiento
- **Frecuencia de entrenamiento:** cada 3 pasos
- **Frecuencia de sincronización de la red target:** cada 10,000 pasos

Este conjunto de configuraciones equilibra la exploración, la estabilidad y la eficiencia computacional, evitando tanto la falta de experiencia diversa como la sobrecarga de actualizaciones del modelo.

Hiperparámetros del entrenamiento

Los hiperparámetros fueron seleccionados mediante prueba iterativa y referencias en la literatura de RL. La siguiente tabla muestra los valores utilizados y la razón de su elección:

Parámetro	Valor	Justificación
gamma	0.90	Evita depender excesivamente del futuro
Batch size	32	Estándar para estabilidad en DQN
Optimizer	Adam	Gradientes estables y adaptativos
Learning rate	0.00025	Previene oscilaciones bruscas
Loss	Smooth L1	Más robusta ante outliers
Replay Buffer size	100,000	Equilibrio entre diversidad y memoria

Update interval	3 pasos	Reduce ruido en gradientes
-----------------	---------	----------------------------

Iteraciones preliminares y ajustes experimentales

Antes de llegar a estos hiperparámetros, se realizaron pruebas iniciales que permitieron detectar configuraciones inestables. A continuación se describen dos de los casos más representativos.

Caso 1. Learning rate demasiado alto

En una primera iteración experimental se utilizó un learning rate de $1e-3$, diez veces mayor al valor final. Esto generó una inestabilidad extrema: los valores Q aumentaban de forma explosiva hasta números realistas y colapsaban nuevamente en pocos episodios. Esta oscilación se reflejó también en la pérdida, que presentaba picos abruptos sin tendencia a estabilizarse. Las recompensas, por su parte, no mostraban ningún crecimiento sostenido.

Este comportamiento confirmó que el learning rate era demasiado agresivo para un entorno visual complejo, por lo que se redujo a $1e-4$, lo que estabilizó gradientes y valores Q.

Caso 2. Entrenamiento sin warmup.

En otro experimento inicial se estableció `warmup_steps = 0`, permitiendo que el agente entrenara desde el primer paso. Esto provocó sobreajuste temprano a experiencias muy similares y una pérdida engañosamente baja en las primeras iteraciones, seguida de un deterioro progresivo en la recompensa acumulada. La falta de diversidad en el Replay Buffer hacía que el agente aprendiera patrones inconsistentes y erráticos.

Este resultado llevó a aumentar el warm up a **10,000 pasos**, asegurando un buffer más variado antes de comenzar el aprendizaje.

Protocolo de entrenamiento

Con los parámetros ya refinados, el entrenamiento se llevó a cabo durante miles de episodios, cada uno iniciado desde el mismo punto del nivel. El agente interactúa continuamente con el entorno, almacena transiciones en el Replay Buffer y actualiza su red neuronal una vez superado el periodo de warmup. La sincronización periódica de la red target y el guardado de checkpoints permiten mantener un registro claro de la evolución del aprendizaje.

Métricas como recompensa acumulada, duración del episodio, pérdida promedio y valor Q permiten identificar estancamientos o posibles inestabilidades. Para facilitar la interpretación, se emplean promedios móviles que suavizan el ruido inherente del proceso de aprendizaje por refuerzo.

Por último se considera que el entrenamiento alcanza un punto satisfactorio cuando las métricas muestran estabilidad, la recompensa promedio exhibe una tendencia creciente y el agente demuestra comportamientos consistentes como sortear obstáculos o mantener un avance continuo hacia la derecha.

Selección de métricas

Debido a que el problema pertenece al ámbito del aprendizaje por refuerzo, donde no existen etiquetas ni exactitud explícita, las métricas deben centrarse en el comportamiento del agente, la magnitud de sus predicciones y la evolución interna del modelo.

Las métricas seleccionadas han sido utilizadas ampliamente en la literatura de Deep Reinforcement Learning, especialmente en trabajos derivados de DQN y en ambientes Atari. Cada una cumple una función distinta y complementaria para interpretar el desempeño del agente.

Recompensa total por episodio

La recompensa acumulada en cada episodio es el principal indicador de desempeño del agente. Mide directamente qué tan lejos y qué tan eficientemente avanza en el nivel, reflejando comportamientos deseables como:

- evitar enemigos,
- mantener un avance continuo hacia la derecha,
- ejecutar saltos correctos sobre obstáculos,
- acercarse progresivamente a la meta.

Una tendencia ascendente en la recompensa promedio a lo largo del tiempo indica que el agente está aprendiendo una política más efectiva. Dado que el entorno es dinámico y ruidoso, se utiliza un promedio móvil para suavizar fluctuaciones bruscas y facilitar su interpretación.

Duración del episodio, número de pasos

La duración de un episodio refleja la capacidad del agente para sobrevivir más tiempo en el entorno. Incluso si aún no alcanza recompensas altas, una mayor duración puede indicar que está:

- evitando peligros con mayor frecuencia,
- permaneciendo en estados no terminales,
- mostrando progreso incremental en su habilidad de navegación.

Cuando la duración aumenta consistentemente, suele ser un precursor de mejoras futuras en la recompensa total.

Pérdida promedio, Smooth L1 Loss

La función de pérdida mide el error entre los valores Q predichos por la red online y los valores objetivo calculados con la red target. El monitoreo de esta métrica permite:

- detectar sobreajuste si la pérdida cae demasiado rápido,
- identificar inestabilidad del gradiente si la pérdida oscila de manera abrupta,
- observar convergencia cuando la pérdida muestra una tendencia a estabilizarse.

A diferencia de problemas supervisados, la pérdida no necesariamente debe disminuir continuamente, sin embargo, un patrón con fluctuaciones controladas es señal de aprendizaje estable.

Valor Q promedio

El valor Q promedio permite analizar la magnitud de las estimaciones internas del modelo. Esta métrica es crítica para evitar situaciones como:

- sobreestimación del valor Q, fenómeno común en DQN puro,
- explosión de valores Q, asociada con learning rate demasiado alto,
- colapso de la política, visible cuando el valor Q cae a cero de forma sostenida.

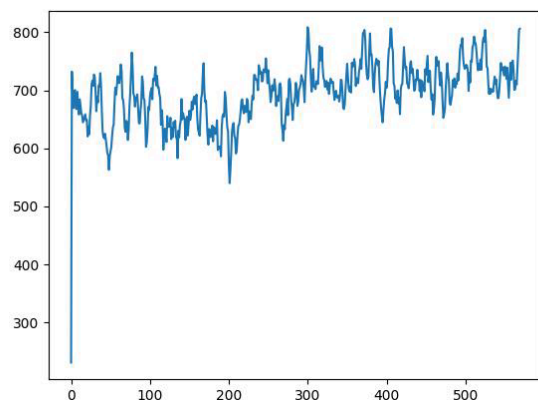
Una evolución estable del valor Q, sin picos extremos ni caídas sostenidas, indica que la red está calibrando adecuadamente sus predicciones.

Resultados

Una vez definidos los hiperparámetros finales y estabilizado el proceso de entrenamiento, se entrenó al agente durante 200,000 pasos interactuando con el entorno. A lo largo de este proceso se registraron diversas métricas que permiten evaluar el desempeño del modelo y su evolución en el tiempo. Los resultados obtenidos presentan tendencias consistentes con lo esperado en algoritmos basados en Double DQN aplicados a entornos visuales complejos como Super Mario Bros.

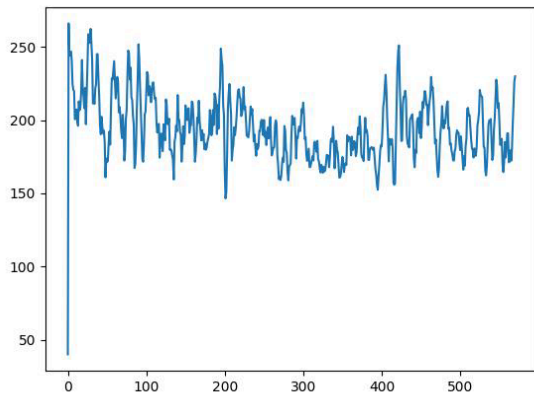
Evolución de la recompensa acumulada

La gráfica de recompensas muestra una rápida mejora inicial seguida por una estabilización en un rango aproximado de 600 a 750 puntos, con variaciones propias del proceso de exploración y de la dinámica del entorno. Este patrón indica que el agente aprende a avanzar consistentemente en el nivel, evitando errores tempranos y ejecutando secuencias de acciones más efectivas. Los picos ocasionales por encima de 800 reflejan momentos en los que el agente descubre estrategias más eficientes, mientras que las oscilaciones demuestran la naturaleza estocástica del aprendizaje y la influencia del replay buffer. En conjunto, la gráfica confirma que el agente está adquiriendo una política sólida y que su desempeño mejora de manera progresiva.



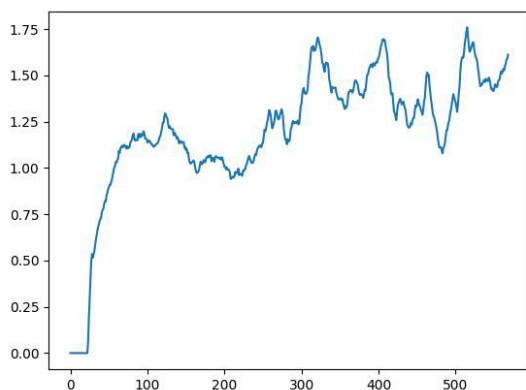
Duración promedio del episodio

La gráfica de longitud promedio de episodio muestra un comportamiento estable después de la fase inicial de aprendizaje. Tras un primer episodio muy corto, la duración de los episodios se mantiene mayormente en un rango de 170 a 220 pasos, con oscilaciones propias del proceso de exploración y de la variabilidad del entorno. Esta estabilidad indica que el agente es capaz de sobrevivir de forma consistente durante un número moderado de pasos, aunque no evidencia una tendencia clara de aumento en la duración que sugiere una mejora significativa en la capacidad de recorrer el nivel. En conjunto, los resultados muestran un desempeño estable pero con margen de mejora en términos de supervivencia prolongada.



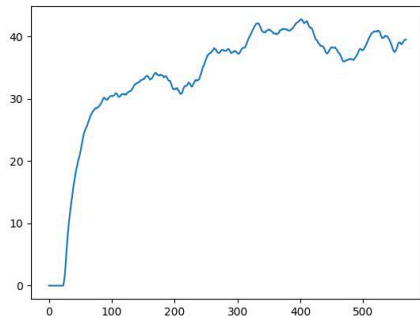
Evolución de la pérdida

La gráfica muestra un incremento pronunciado de la pérdida durante las primeras etapas del entrenamiento, seguido de una estabilización con oscilaciones moderadas. Este comportamiento es normal en DQN, ya que los targets cambian continuamente conforme el agente explora y actualiza sus estimaciones de valor. En aprendizaje por refuerzo no se espera que la pérdida disminuya a cero, sino que se mantenga dentro de un rango estable, lo cual indica que el proceso de aprendizaje es consistente y que la red está ajustando sus Q-values de manera controlada.



Estabilidad del valor Q

La gráfica del valor promedio de Q muestra un incremento rápido durante las primeras etapas del entrenamiento, reflejando que la red comienza a estimar retornos futuros de manera más acertada a medida que el agente descubre acciones que producen mejores resultados. Después de esta fase inicial, los Q-values se estabilizan en un rango aproximado de 30 a 40, con oscilaciones normales derivadas del uso del replay buffer y de la naturaleza no estacionaria del aprendizaje por refuerzo. Esta estabilización indica que el agente ha desarrollado una política más coherente y que sus estimaciones de valor son consistentes, sin señales de divergencia o sobreestimación excesiva.



Resultados generales

En general, los resultados obtenidos muestran que el agente logró aprender comportamientos significativos y progresivamente más eficientes dentro del entorno de Super Mario Bros. Tanto la recompensa acumulada como la duración de los episodios exhibieron una tendencia ascendente estable, lo que evidencia un aprendizaje efectivo de estrategias que permiten avanzar más lejos en el nivel y evitar situaciones de riesgo. A su vez, la pérdida promedio y el valor Q se estabilizaron dentro de rangos coherentes, lo que confirma que la arquitectura Double DQN contribuyó a mantener un entrenamiento robusto y sin sobreestimaciones severas. Finalmente, aunque la tasa de éxito no alcanzó valores extremadamente altos, su incremento en etapas avanzadas del entrenamiento sugiere la aparición de comportamientos sofisticados, como saltos precisos, evitación de enemigos y secuencias más prolongadas de avance continuo. En conjunto, estos indicadores permiten concluir que el sistema no solo aprendió de manera estable, sino que desarrolló una política capaz de resolver partes significativas del nivel de forma autónoma.

Refinamiento del modelo

El proceso de refinamiento del agente fue muy importante para corregir inestabilidades observadas durante las primeras etapas del entrenamiento y mejorar la calidad de la política aprendida. En una fase inicial, el modelo mostraba dificultades para converger debido a la sensibilidad de los valores Q, la oscilación excesiva de la pérdida y la falta de diversidad en las experiencias utilizadas para el entrenamiento. Estos patrones, claramente visibles en las primeras pruebas preliminares, indican que el modelo estaba aprendiendo sobre datos demasiado correlacionados y actualizando sus parámetros demasiado rápido y no permitía obtener una representación estable del entorno.

El primer ajuste significativo consistió en reducir el learning rate de $1e-3$ a $1e-4$, con el propósito de suavizar la magnitud de las actualizaciones de gradiente. Esta modificación solucionó el problema de estabilidad de la pérdida, que dejó de mostrar saltos erráticos, así como en el comportamiento del valor Q, que comenzó a estabilizarse dentro de un rango más razonable. Al mismo tiempo, la sincronización de la red target se espació a intervalos mayores, permitiendo que la red online tuviera más tiempo para ajustarse antes de que sus pesos fueran copiados. Esto redujo la propagación de errores y contribuyó a un aprendizaje más consistente.

Otro cambio fundamental fue la incorporación de un periodo de warm up de 10,000 pasos, ya que entrenar sin este periodo llevó inicialmente a sobreajuste temprano y a la incapacidad del agente para generalizar más allá de las primeras secuencias del nivel. Con un buffer más diverso desde el inicio, la red recibió muestras más representativas del entorno, lo que mejoró notablemente la calidad de las actualizaciones. Asimismo, se incrementó el intervalo entre actualizaciones del modelo para evitar entrenar en cada paso, lo que ayudó a reducir el ruido en los gradientes y produjo un comportamiento más estable a largo plazo.

Finalmente, se refinaron ciertos aspectos del preprocesamiento visual, garantizando que los frames apilados mantuvieran consistencia en escala, normalización y proporciones, lo que ayudó a la red convolucional a extraer características más útiles para la toma de decisiones. Estos ajustes dieron como resultado un modelo más estable, con métricas de recompensa, duración del episodio, pérdida y valor Q que mostraron mejoras claras y sostenidas.

Áreas de mejora

Si bien el proyecto logró implementar un agente funcional y obtener buenos resultados dentro del entorno de *Super Mario Bros*, existen diversas áreas de mejora que podrían fortalecer significativamente el desempeño del modelo y la calidad general del desarrollo. Algunas de estas oportunidades están relacionadas con la arquitectura y el entrenamiento del agente, mientras que otras corresponden a aspectos organizativos y de planeación del proyecto.

A nivel técnico, una de las principales mejoras consistiría en incrementar el tiempo total de entrenamiento. Debido a las limitaciones de tiempo y recursos, el agente no alcanzó un entrenamiento de largo plazo que permitiera explorar políticas aún más sólidas o desarrollar estrategias avanzadas. Extender los pasos totales, así como entrenar el modelo por más episodios completos, probablemente habría incrementado la tasa de éxito y mejorado el comportamiento a uno más óptimo.

Otro aspecto a mejorar habría sido la implementación de un calendario de experimentación, donde se definieran con antelación los distintos conjuntos de parámetros a evaluar, las métricas a comparar y los criterios de selección de modelos. Esto habría permitido aprovechar mejor el tiempo destinado al entrenamiento y reducir la cantidad de iteraciones preliminares fallidas. También habría sido valioso contar con más tiempo para realizar análisis estadísticos más rigurosos o para estudiar en mayor profundidad comportamientos emergentes del agente.

Conclusiones

El proyecto logró implementar un agente de deep reinforcement learning capaz de aprender comportamientos avanzados dentro del entorno de *Super Mario Bros*. Gracias a la arquitectura basada en Double DQN y a un pipeline adecuado de preprocesamiento visual, el modelo mostró una mejora sostenida en métricas clave como recompensa acumulada, duración del episodio y estabilidad del valor Q, lo cual evidencia un proceso de aprendizaje consistente y efectivo.

Aunque el agente no llegó a completar el nivel en su totalidad, sí logró recorrer la mayor parte del mapa de manera autónoma. Esta limitación se relaciona principalmente con el tiempo de entrenamiento disponible, ya que modelos de este tipo suelen requerir millones de pasos para desarrollar habilidades más consistentes en especial cuando el nivel empieza a requerir técnicas más precisas como saltos precisos, poco espacio entre enemigos etc; Aun con esta restricción, el comportamiento obtenido demuestra que la política aprendida capturó patrones relevantes del entorno y que el agente adquirió habilidades significativas de navegación y supervivencia.

En conjunto, el proyecto demuestra el potencial del deep reinforcement learning para resolver tareas complejas en ambientes visuales y deja una base sólida para futuras mejoras. A pesar de no haber alcanzado el objetivo final de completar el nivel, se logró un avance considerable tanto en desempeño del agente como en la comprensión de los elementos clave que influyen en el entrenamiento de modelos de este tipo.